

```

    move_profile.append(score(game2))
    if depth < maxdepth:
        possible_moves2 = possible_moves.copy()
        possible_moves2.remove(move)
        move_profile.append(generate_tree(possible_moves2, depth +
1, maxdepth, game2))
    else:
        move_profile.append([])
    tree.append(move_profile)
return(tree)

```

[Listing 10-5](#): A function that generates a game tree, including child moves and game scores

We can call this again as follows:

```

allpossible = [[(4,4),(4,3)],[(4,1),(5,1)]]
thetree = generate_tree(allpossible,0,1,[])
print(thetree)

```

We see the following results:

```

[[[(4, 4), (4, 3)], [0, 0], [[[4, 1), (5, 1)], [0, 0], []]]], [[(4, 1), (5, 1)],
[0, 0], \[[[(4, 4), (4, 3)], [0, 0], []]]]]

```

You can see that each entry in this tree is a full move profile, consisting of a move (like [(4,4),(4,3)]), a score (like [0,0]), and a (sometimes empty) list of children.

Winning a Game

We're finally ready to create a function that can play dots and boxes well. Before we write the code, let's consider the principles behind it. Specifically, how is it that we, as humans, play dots and boxes well? More generally, how is it that we go about winning any strategic game (like chess or tic-tac-toe)? Every game has unique rules and features, but there's a general way to choose a winning strategy based on an analysis of the game tree.

The algorithm we'll use for choosing a winning strategy is called *minimax* (a combination of the words *minimum* and *maximum*), so called because while we're trying to maximize our score in the game, our